NAME:ARJA TEJASWI
BRANCH/SECTION:CSE-1
ROLL.NO:26000008

PYTHON CASE STUDIES

10 Complete Beginner-Friendly Solutions

Problem statements • Objectives • Assumptions • Algorithms • Flowcharts • Pseudocode • Python code • Test cases

Prepared using the assigned introductory Python syllabus

About this document

The programs use beginner-level Python constructs: variables, data types, operators, conditional statements, loops, strings, lists, nested lists, tuples, tuple unpacking and basic built-in operations. Flowcharts are provided as clear step-by-step flow sequences so they can be redrawn using standard flowchart symbols.

1. Tuple-Based Transaction Audit

Problem statement

Process transaction tuples containing an identifier, amount and transaction type. Detect duplicate identifiers, invalid amounts and suspicious consecutive transactions.

Objectives

• Detect duplicate transaction IDs.
• Reject zero or negative amounts.
• Identify consecutive transactions using the same ID.

Assumptions

Each tuple is (ID, amount, type). Amounts must be positive and types must be CREDIT or DEBIT. Consecutive transactions with the same ID are treated as suspicious.

Algorithm

1. Read the transaction tuples.
2. Check each ID against previously seen IDs.
3. Validate the amount and transaction type.
4. Compare each transaction ID with the previous one.
5. Display audit messages.

Flowchart

Start → Read transactions → Check duplicate ID, amount and type → Check consecutive repeated IDs → Display audit results → Stop.

Pseudocode

```
START
READ transactions
FOR each transaction
    UNPACK ID, amount, type
    CHECK duplicate ID
    CHECK amount and type
    CHECK consecutive repeated ID
DISPLAY results
STOP
```

Python source code

```
transactions = [
    (101, 500, "CREDIT"),
    (102, 200, "DEBIT"),
    (101, -50, "DEBIT"),
    (103, 100, "CREDIT")
]

ids = []

for t in transactions:
    tid, amount, kind = t

    if tid in ids:
        print(tid, "Duplicate ID")
    else:
        ids.append(tid)

    if amount <= 0:
        print(tid, "Invalid amount")

    if kind != "CREDIT" and kind != "DEBIT":
        print(tid, "Invalid type")
```

```
for i in range(1, len(transactions)):
    if transactions[i][0] == transactions[i-1][0]:
        print("Suspicious consecutive transactions:", transactions[i][0])
```

Test cases

Input	Expected result
(101, 500, CREDIT)	Valid transaction
(101, -50, DEBIT), after ID 101	Duplicate ID and invalid amount
Two consecutive records with ID 101	Suspicious sequence



2. Student Group Formation with Score Balancing

Problem statement

Divide students into groups of a specified size and compare the total scores of the groups.

Objectives

• Divide students into groups.
• Calculate each group's total score.
• Compare group totals.

Assumptions

Scores are non-negative integers. Scores are sorted in descending order and divided into consecutive groups of the specified size. The last group may be smaller. This simple method does not guarantee optimal balance.

Algorithm

1. Read scores and group size.
2. Sort scores in descending order.
3. Divide the sorted scores into groups of the given size.
4. Calculate each group's total.
5. Display group totals and their difference.

Flowchart

Start → Read scores and group size → Sort scores → Form groups → Calculate totals and difference → Display results → Stop.

Pseudocode

```
START
READ scores and group size
SORT scores in descending order
CREATE groups
DIVIDE scores into groups of specified size
CALCULATE each group total
DISPLAY groups and difference
STOP
```



Python source code

```
scores = [90, 85, 80, 75, 70, 65]
size = 3

scores.sort(reverse=True)
groups = []
totals = []

for i in range(0, len(scores), size):
    groups.append(scores[i:i+size])
    totals.append(sum(scores[i:i+size]))

for i in range(len(groups)):
    print("Group", i+1, groups[i], "Total:", totals[i])

print("Difference:", max(totals) - min(totals))
```

Test cases

Scores / group size	Expected result
[90, 85, 80, 75] / 2	Groups [90, 85] and [80, 75]; totals 175 and 155
[60, 70, 80] / 2	Groups [80, 70] and [60]; totals 150 and 60

3. Smart Canteen Order and Stock Management

Problem statement

Process customer orders, calculate bills, update stock, reject unavailable quantities and summarize sales.

Objectives

• Calculate bills for accepted orders.
• Prevent stock from becoming negative.
• Display total sales and remaining stock.

Assumptions

Each order is (item index, quantity). Item indexes start at zero. Orders with invalid indexes or non-positive quantities are rejected. Stock cannot become negative.

Algorithm

1. Store item names, prices and stock.
2. Read each order.
3. Check whether the item index and quantity are valid.
4. If stock is sufficient, calculate the bill and update stock.
5. Display total sales and remaining stock.

Flowchart

Start → Read menu, stock and orders → Validate order → Check stock → Accept and update stock OR reject → Display sales and stock → Stop.

Pseudocode

```
START
READ menu, prices, stock and orders
FOR each order
    IF item index and quantity are valid
        IF quantity is available
            CALCULATE bill and update stock
            ADD bill to sales
        ELSE DISPLAY order rejected
    ELSE DISPLAY invalid order
DISPLAY sales and remaining stock
STOP
```

Python source code

```
items = ["Idli", "Dosa", "Juice"]
prices = [30, 50, 25]
stock = [20, 10, 15]
orders = [(0, 2), (1, 3), (2, 20)]
sales = 0

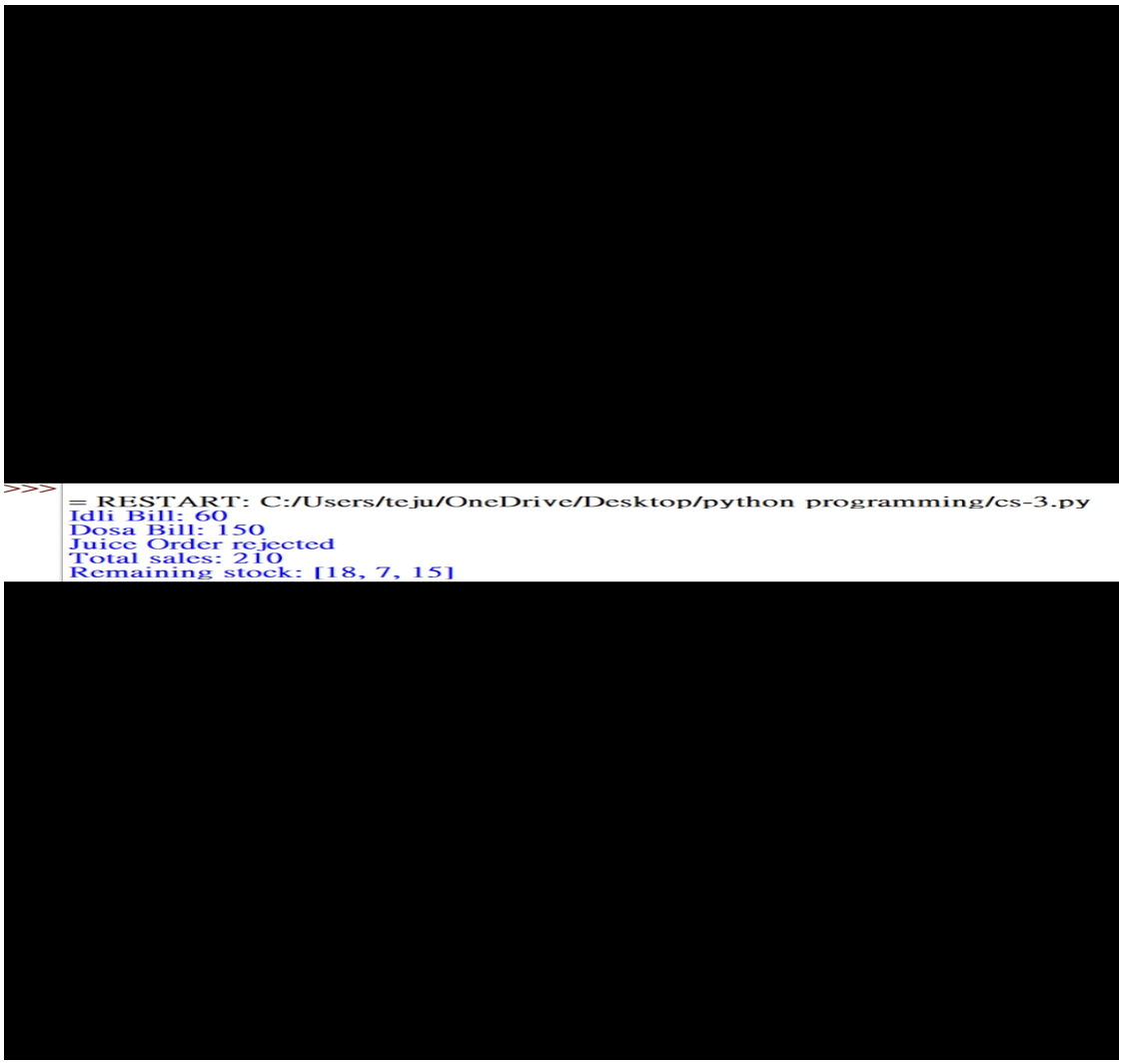
for item, qty in orders:
    if 0 <= item < len(items) and qty > 0:
        if qty <= stock[item]:
            bill = prices[item] * qty
            stock[item] -= qty
            sales += bill
            print(items[item], "Bill:", bill)
        else:
            print(items[item], "Order rejected")
    else:
        print("Invalid order")

print("Total sales:", sales)
print("Remaining stock:", stock)
```

Test cases

Order	Expected result
(0, 2)	Idli bill = Rs. 60
(1, 3)	Dosa bill = Rs. 150

Order	Expected result
(2, 20)	Rejected; only 15 juices available
All orders	Sales = Rs. 210; stock = [18, 7, 15]



4. Library Borrowing and Fine Reconciliation

Problem statement

Analyse borrowing records, calculate fines for late returns, identify overdue books and determine book availability.

Objectives

• Calculate fines for delayed days.
• Identify overdue records.
• Count borrowed and available books.

Assumptions

Each tuple is (book ID, borrowed days, delay days). The fine is Rs. 5 per delayed day. Records represent currently borrowed books. Total book count is known.

Algorithm

1. Read borrowing records and total book count.
2. Calculate the fine for each record.
3. Display records with a delay greater than zero.
4. Count borrowed books and calculate available books.
5. Display total fines and availability.

Flowchart

Start → Read records and total books → Calculate fine → Check overdue records → Calculate availability → Display results → Stop.

Pseudocode

```
START
READ records and total books
FOR each record
    CALCULATE fine = delay × rate
    IF delay > 0 DISPLAY overdue book
COUNT borrowed books
CALCULATE available books
DISPLAY total fine and availability
STOP
```

Python source code

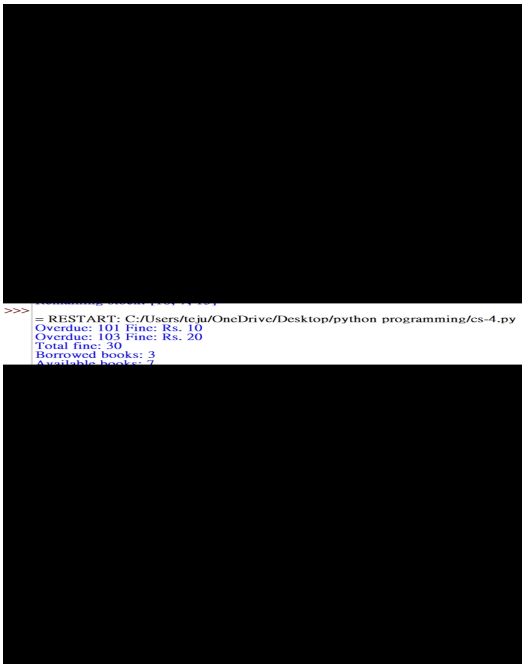
```
records = [(101, 7, 2), (102, 5, 0), (103, 10, 4)]
total_books = 10
fine_total = 0

for book, days, delay in records:
    fine = delay * 5
    fine_total += fine

    if delay > 0:
        print("Overdue:", book, "Fine: Rs.", fine)

borrowed = len(records)
available = total_books - borrowed

print("Total fine:", fine_total)
print("Borrowed books:", borrowed)
print("Available books:", available)
```



Test cases

Record	Expected result
(101, 7, 2)	Fine = Rs. 10
(102, 5, 0)	No fine
All three records	Total fine = Rs. 30; 7 books available

5. Campus Transport Allocation

Problem statement

Calculate buses required for each route using passenger demand and bus capacity, and identify unserved passengers when the available bus count is limited.

Objectives

• Calculate required buses.
• Compare passenger demand with available seats.
• Identify unserved passengers.

Assumptions

Each route is (route name, passenger count, available buses). All buses have equal capacity. Required buses are rounded up to the next whole bus.

Algorithm

1. Read route demands, bus capacity and available buses.
2. Calculate required buses using division and remainder.
3. Calculate available seats.
4. Calculate unserved passengers.
5. Display route results.

Flowchart

Start → Read route demand and capacity → Calculate required buses → Calculate available seats → Calculate unserved passengers → Display results → Stop.

Pseudocode

```
START
READ route demands, capacity and available buses
FOR each route
    CALCULATE required buses
    CALCULATE available seats
    CALCULATE unserved passengers
DISPLAY route results
STOP
```

Python source code

```
routes = [("Route A", 120, 2), ("Route B", 75, 2)]
capacity = 50

for name, passengers, buses in routes:
    required = passengers // capacity
    if passengers % capacity != 0:
        required += 1

    seats = buses * capacity
    unserved = max(0, passengers - seats)

    print(name, "Required buses:", required)
    print("Unserved passengers:", unserved)
```



Test cases

Route	Expected result
A: 120 passengers, 2 buses	3 required; 20 unserved
B: 75 passengers, 2 buses	2 required; 0 unserved

6. Manufacturing Quality Control Matrix

Problem statement

Analyse production quantities and defective units in a nested list, calculate defect percentages and identify batches exceeding a quality threshold.

Objectives

• Calculate defect percentages.
• Identify batches exceeding the allowed limit.
• Calculate total defective units.

Assumptions

Each row is [produced, defective]. Production quantity must be greater than zero and defective units cannot exceed production. The acceptable defect threshold is 5%.

Algorithm

1. Read the production matrix and threshold.
2. For each batch, unpack produced and defective quantities.
3. Calculate defect percentage.
4. Compare it with the threshold.
5. Add defective units and display the total.

Flowchart

Start → Read production matrix → Calculate defect percentage → Compare with 5% threshold → Add defects → Display batch status and total → Stop.

Pseudocode

```
START
READ batches and threshold
FOR each batch
    UNPACK produced and defective
    CALCULATE defect percentage
    IF percentage > threshold DISPLAY failed batch
    ADD defects to total
DISPLAY total defects
STOP
```

Python source code

```
batches = [[1000, 20], [500, 40], [800, 24]]
threshold = 5
total_defects = 0

for i in range(len(batches)):
    produced, defective = batches[i]
    total_defects += defective
    percentage = defective / produced * 100

    print("Batch", i+1, "Defect:", percentage, "%")

    if percentage > threshold:
        print("Batch exceeds threshold")

print("Total defective units:", total_defects)
```



Test cases

Batch	Expected result
1000 produced, 20 defective	2%; acceptable
500 produced, 40 defective	8%; exceeds threshold
800 produced, 24 defective	3%; acceptable
All batches	84 defective units

7. Nested List Examination Seating Validation

Problem statement

Validate an examination seating matrix for duplicate roll numbers, reserved seats, empty seats and invalid allocations.

Objectives

• Detect duplicate roll numbers.
• Check reserved and empty positions.
• Identify invalid roll numbers.

Assumptions

0 represents an empty seat, -1 represents a reserved seat and positive values represent roll numbers. Reserved positions are provided as (row, column), using zero-based indexes.

Algorithm

1. Read the seating matrix and reserved positions.
2. Visit each seat using nested loops.
3. Check whether reserved positions contain -1.
4. Identify empty seats and invalid negative values.
5. Detect duplicate positive roll numbers.

Flowchart

Start → Read seating matrix and reserved positions → Visit each seat → Check reserved, empty and invalid values → Check duplicates → Display errors → Stop.

Pseudocode

```
START
READ seating matrix and reserved positions
CREATE empty roll number list
FOR each row and column
    CHECK reserved and empty seats
    CHECK invalid roll numbers
    CHECK duplicates
DISPLAY errors
STOP
```

Python source code

```
seats = [[101, 102, 0], [103, 101, -1]]
reserved = [(1, 2)]
rolls = []

for i in range(len(seats)):
    for j in range(len(seats[i])):
        roll = seats[i][j]

        if (i, j) in reserved and roll != -1:
            print("Reserved seat error:", i, j)

        if roll == 0:
            print("Empty seat:", i, j)
        elif roll == -1:
            if (i, j) not in reserved:
                print("Invalid reserved position:", i, j)
        elif roll < 0:
            print("Invalid roll number:", roll)
        elif roll in rolls:
            print("Duplicate roll number:", roll)
        else:
            rolls.append(roll)
```

Test cases

Input condition	Expected result
Roll 101 occurs twice	Duplicate roll number
Seat contains 0	Empty seat

Input condition	Expected result
Reserved seat contains 101	Reserved seat error
-1 at an unreserved position	Invalid reserved position



8. Water Distribution Fairness Analysis

Problem statement

Analyse area-wise water supply across several days, calculate shortages against demand and identify the most underserved area.

Objectives

• Calculate daily water shortages.
• Compare supply with demand.
• Identify the area with the highest total shortage.

Assumptions

Each matrix row represents an area and each column represents a day. Demand is constant for each area. Shortage is never negative.

Algorithm

1. Read area names, demands and supply matrix.
2. For each area, compare daily supply with demand.
3. Add daily shortages to get the area total.
4. Find the area with the maximum total shortage.
5. Display results.

Flowchart

Start → Read demand and supply matrix → Calculate daily shortages → Calculate total shortage per area → Find maximum → Display underserved area → Stop.

Pseudocode

```
START
READ areas, demands and supplies
FOR each area
    SET total shortage to zero
    FOR each day's supply
        CALCULATE shortage
        ADD shortage to total
DISPLAY area with maximum shortage
STOP
```

Python source code

```
areas = ["Area A", "Area B", "Area C"]
demand = [100, 120, 80]
supply = [[90, 80, 100],
          [100, 90, 110],
          [70, 60, 80]]

totals = []

for i in range(len(areas)):
    shortage = 0

    for water in supply[i]:
        shortage += max(0, demand[i] - water)

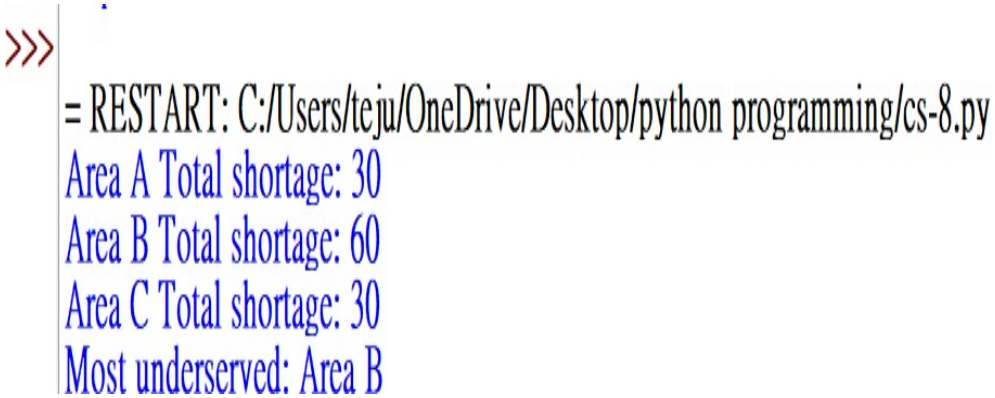
    totals.append(shortage)
    print(areas[i], "Total shortage:", shortage)

index = totals.index(max(totals))
print("Most underserved:", areas[index])
```

Test cases

Area	Expected result
A: demand 100; supply 90, 80, 100	Shortage = 30
B: demand 120; supply 100, 90, 110	Shortage = 60
C: demand 80; supply 70, 60, 80	Shortage = 30

Area	Expected result
All areas	Area B is most underserved



9. Multi-Product Price Fluctuation Analysis

Problem statement

Analyse historical product prices, calculate percentage changes, identify the maximum fluctuation and flag products exceeding a threshold.

Objectives

• Calculate percentage price changes.
• Find the largest absolute fluctuation.
• Flag products exceeding the threshold.

Assumptions

Each product has at least two positive historical prices. Percentage change is measured from the first to the last price. A 10% absolute change is the threshold.

Algorithm

1. Read product names and historical prices.
2. Calculate percentage change from first to last price.
3. Flag products whose absolute change exceeds the threshold.
4. Track the largest absolute fluctuation.
5. Display results.

Flowchart

Start → Read historical prices → Calculate percentage changes → Compare with threshold → Track maximum fluctuation → Display results → Stop.

Pseudocode

```
START
READ products and threshold
SET maximum fluctuation to zero
FOR each product
    CALCULATE percentage change
    IF absolute change > threshold DISPLAY flagged product
    UPDATE maximum fluctuation
DISPLAY maximum fluctuation
STOP
```

Python source code

```
products = [
    ("Rice", [50, 55, 60]),
    ("Milk", [40, 42, 41]),
    ("Oil", [100, 120, 130])
]
threshold = 10
maximum = -1

for name, prices in products:
    first = prices[0]
    last = prices[-1]
    change = (last - first) / first * 100

    print(name, "Change:", change, "%")

    if abs(change) > threshold:
        print("Flagged:", name)

    if abs(change) > maximum:
        maximum = abs(change)
        product_name = name

print("Maximum fluctuation:", product_name, maximum, "%")
```

Test cases

Product	Expected result
Rice: Rs. 50 to Rs. 60	20%; flagged

Product	Expected result
Milk: Rs. 40 to Rs. 41	2.5%; not flagged
Oil: Rs. 100 to Rs. 130	30%; flagged and maximum

```
>>>
= RESTART: C:/Users/teju/OneDrive/Desktop/python programming/cs-9.py
Rice Change: 20.0 %
Flagged: Rice
Milk Change: 2.5 %
Oil Change: 30.0 %
Flagged: Oil
Maximum fluctuation: Oil 30.0 %
```

10. Tuple-Based Course Registration Validation

Problem statement

Validate course registration tuples, identify duplicate registrations, check invalid course combinations and report students violating the rules.

Objectives

• Identify duplicate registrations.
• Check prohibited course combinations.
• Display students who violate registration rules.

Assumptions

Each tuple is (student ID, course). A student cannot register for the same course twice and cannot register for both AI and ML. The example checks students 101 and 102.

Algorithm

1. Read registration tuples.
2. Check each tuple against previously accepted registrations.
3. Report duplicate pairs.
4. Collect each student's courses.
5. Report students registered for both AI and ML.

Flowchart

Start → Read registration tuples → Check duplicate pairs → Collect courses by student → Check prohibited combinations → Display violations → Stop.

Pseudocode

```
START
READ registrations
CREATE empty valid registration list
FOR each registration
    CHECK duplicate pair
    IF new pair ADD to valid list
FOR each student
    CHECK prohibited course combination
DISPLAY violations
STOP
```

Python source code

```
registrations = [
    (101, "Python"),
    (101, "AI"),
    (102, "ML"),
    (101, "Python"),
    (102, "AI")
]

valid = []

for reg in registrations:
    student, course = reg

    if reg in valid:
        print("Duplicate registration:", reg)
    else:
        valid.append(reg)

for student in [101, 102]:
    courses = []

    for sid, course in valid:
        if sid == student:
            courses.append(course)

    if "AI" in courses and "ML" in courses:
        print("Invalid combination for student:", student)
```

Test cases

Input condition	Expected result
(101, Python) appears twice	Duplicate registration
Student 102 registers for AI and ML	Invalid combination for student 102
Student 101 registers for Python and AI	Valid combination

